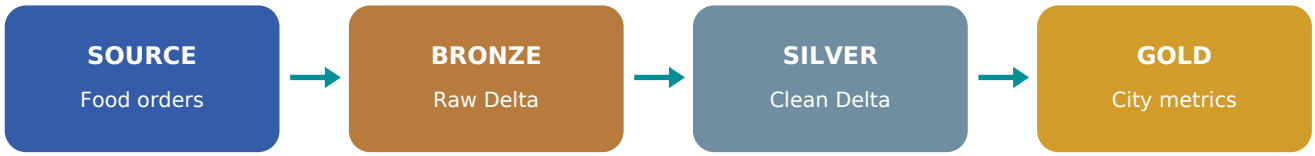
# DATABRICKS STEP-BY-STEP HANDS-ON LAB MANUAL

Workspace • Notebooks • Compute • PySpark • Delta Lake
Medallion Architecture • Unity Catalog • Lakeflow Jobs •
Troubleshooting




Unity Catalog governs tables | Lakeflow Jobs orchestrates steps

**Practical example:** Food delivery order analytics, from raw orders to city-level reporting.

**Audience:** Beginners, data engineers, architects, trainers, and platform operators.

**Edition:** November 2026 | Databricks Free Edition (serverless) and supported paid workspaces.

> **NOTE** Use synthetic training data only. Menu names can differ by cloud, account type, or workspace release. Never paste real credentials into notebooks.

# How to use this manual

Follow the labs in order. The same food-order dataset is created once, refined using PySpark, stored in Delta tables, queried using SQL, and scheduled as a multi-task job. Every lab contains navigation, actions, commands, expected results, and checkpoints.

## Before you begin

01. Open a Databricks workspace and confirm that your user can create notebooks.
02. Use a Unity Catalog-enabled workspace with a writable workspace catalog or schema.
03. Choose serverless compute for Free Edition. If your organization exposes classic compute, use a permitted existing cluster or create one.
04. Keep the naming convention in the manual so notebook references and tables match.
05. Allocate approximately 5-7 hours for the full path, plus optional troubleshooting drills.

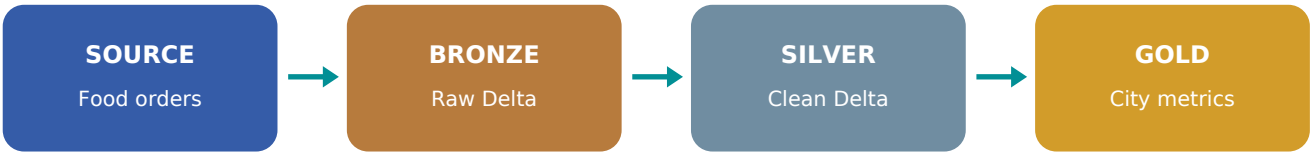
## Lab roadmap

| Lab | Topic | What you will produce |
|---|---|---|
| 01 | Workspace navigation | Folder + notebook |
| 02 | Notebook execution | Python and SQL cells |
| 03 | Compute lifecycle | Compute choice and status checks |
| 04 | PySpark source data | Food orders DataFrame |
| 05 | Bronze / Silver / Gold | Three managed Delta tables |
| 06 | SQL analysis and charts | City revenue queries |
| 07 | Delta time travel | Version history and earlier snapshot |
| 08 | Unity Catalog access | Catalog and privileges inspection |
| 09 | Lakeflow Jobs | Three-task workflow |
| 10 | Job monitoring and repair | Failure diagnosis and repair |
| 11 | Compute / Spark troubleshooting | Operational checks |
| 12 | Capstone and cleanup | Evidence + validation checklist |

## Code conventions

**%python** and **%sql** identify the notebook cell language. In a Python notebook, enter Python without `%python`; enter `%sql` as the first line for a SQL cell. `dbx_food_lab` is the training schema in your current catalog. The manual uses unqualified names such as `dbx_food_lab.orders_bronze`; these resolve within your active catalog.

# Architecture and essential terms




Unity Catalog governs tables | Lakeflow Jobs orchestrates steps

| Term | Meaning in this workshop |
|---|---|
| Workspace | Web-based environment for notebooks, catalog exploration and job operations. |
| Notebook | Executable document with Python/SQL cells, outputs and comments. |
| Compute | Execution resources. Serverless is managed; classic compute exposes cluster configuration. |
| DataFrame | Distributed tabular data structure operated on using Spark. |
| Delta Lake | Table format with transactional writes, schema enforcement and version history. |
| Bronze | Raw records persisted with minimal transformation. |
| Silver | Validated, de-duplicated records and derived fields. |
| Gold | Business-ready aggregates for dashboards and downstream consumption. |
| Unity Catalog | Catalog, schema, tables, permissions, ownership and discovery. |
| Lakeflow Jobs | Orchestration for dependent tasks, schedules, monitoring, retries and repair. |

## What happens during the training?

**01.** Generate eight synthetic food orders in a notebook.

**02.** Store the original eight records as a Bronze Delta table.

**03.** Filter cancelled and invalid orders, remove one duplicate, and produce five Silver records.

**04.** Aggregate the delivered revenue into three Gold city records.

**05.** Inspect Delta history, grant access only when authorized, then orchestrate Bronze -> Silver -> Gold as a job.

# LAB 01 | Navigate the Databricks workspace

**HANDS-ON WALKTHROUGH**

Estimated time: 20 minutes

**Objective:** Identify the main UI elements and create a tidy lab folder.

01. Sign in to your Databricks workspace from your organization link or Databricks Free Edition.
02. Find the left navigation area. Locate **Workspace**, **Catalog**, **Compute** (if available), and **Jobs & Pipelines**.
03. Open **Workspace** and navigate to your home or user folder.
04. Create a folder named `Databricks_Food_Labs` using the folder menu or a create action.
05. Click **+ New > Notebook** (or create a notebook from the folder), name it `01_load_bronze`, and select **Python** if prompted.
06. Locate notebook cells, add-cell controls, and the compute selector / Connect action.

**Expected result:** You can open **Workspace / Databricks_Food_Labs / 01_load_bronze**. The notebook has an editable Python cell.

## Checkpoint

Capture a screenshot showing the notebook name, the cell editor, and the compute connection control. If the folder is not visible, use the workspace search or check your home folder.

# LAB 02 | Run Python and SQL notebook cells

**HANDS-ON WALKTHROUGH**

Estimated time: 25 minutes

**Objective:** Execute cells, switch languages, and inspect results.

01. Open `01_load_bronze`. Use the compute selector to connect to **Serverless** if offered, or an allowed running classic compute.
02. In the first Python cell, enter the sample below and click the Run icon or press **Shift+Enter**.
03. Add a new cell and enter the SQL sample below, beginning with `%sql`.
04. Use **Run All** once both cells work. Observe the cell state and output.
05. Rename the notebook or add Markdown instructions if your environment permits.

**CODE - PYTHON**

```
message = "Hello from Databricks"
print(message)
print("Double of 21:", 21 * 2)
```

**CODE - SQL**

```
%sql
SELECT current_catalog() AS active_catalog,
       current_schema() AS active_schema,
       current_timestamp() AS checked_at;
```

**Expected result:** The Python cell prints a greeting and 42. The SQL cell returns one row with the active catalog, schema and timestamp.

**NOTE** In Free Edition, serverless is the only compute model. If an option is missing, do not try to create a classic cluster; continue with serverless.

# LAB 03 | Understand compute lifecycle

**HANDS-ON WALKTHROUGH**

Estimated time: 25 minutes

**Objective:** Select the right compute and inspect operational status.

## Path A - Serverless (Free Edition and many paid workspaces)

**01.** Open the notebook compute selector and choose **Serverless**, or use the provided default serverless environment.

**02.** Run a one-line Python cell and confirm that the notebook executes.

**03.** Recognize that Databricks manages startup, sizing and scaling for you. You do not manage driver/worker nodes yourself.

**04.** If serverless is unavailable due to a quota or account limitation, record the message. Do not retry continuously.

## Path B - Classic all-purpose compute (only if offered)

**01.** From the sidebar, open **Compute** and select **Create compute**.

**02.** Provide a name such as `food-lab-compute`, choose your organization-approved compute policy, runtime, size and auto-termination setting.

**03.** Click **Create**. Observe the sequence **Pending / Starting -> Running**.

**04.** Attach `01_load_bronze` to the running compute.

**05.** After the workshop, detach/terminate unused classic compute to reduce cost.

| Compute type | Primary use | Lifecycle |
|---|---|---|
| Serverless | Quick notebooks, SQL, supported workflows | Platform manages provisioning and scaling. |
| Classic all-purpose | Interactive development on configured clusters | Start, attach, monitor, auto-terminate. |
| Classic jobs compute | Automated task execution on provisioned resources | Typically created for job run and terminated after. |

**Expected result:** A notebook can execute an expression. If classic compute is available, you can identify its status and event log.

# LAB 04 | Create sample food-order data with PySpark

**HANDS-ON WALKTHROUGH**

Estimated time: 35 minutes

**Objective:** Build a DataFrame and validate data quality problems before storing it.

Continue in the `01_load_bronze` notebook. The raw dataset deliberately includes a cancelled order, a duplicate order and an invalid quantity so that Silver cleansing is observable.

**01.** Add a Python cell. Copy the eight records and schema from the following code block.

**02.** Run the cell. Inspect the DataFrame output and the number of records.

**03.** Review the order status and quantity values: these will determine which records survive the Silver transformation.

**CODE - PYTHON**

```
from pyspark.sql import functions as F

orders = [
 (101,"Chennai","Spice Hub",2,220.0,"Delivered","2026-11-01"),
 (102,"Coimbatore","Cafe Cloud",1,180.0,"Delivered","2026-11-01"),
 (103,"Chennai","Spice Hub",1,180.0,"Cancelled","2026-11-01"),
 (104,"Chennai","Dosa Corner",3,120.0,"Delivered","2026-11-02"),
 (105,"Bengaluru","Green Bowl",2,250.0,"Delivered","2026-11-02"),
 (106,"Coimbatore","Cafe Cloud",1,300.0,"Delivered","2026-11-03"),
 (106,"Coimbatore","Cafe Cloud",1,300.0,"Delivered","2026-11-03"),
 (107,"Chennai","Spice Hub",-1,200.0,"Delivered","2026-11-03")
]
cols = ["order_id","city","restaurant","quantity",
        "unit_price","status","order_date"]
raw_df = spark.createDataFrame(orders, cols)
raw_df = raw_df.withColumn("order_date", F.to_date("order_date"))
display(raw_df)
print("Raw row count:", raw_df.count())
```

**Expected result: 8 rows**. Order 106 appears twice; order 103 is cancelled; order 107 has quantity -1.

## Mini exercise

**CODE - PYTHON**

```
raw_df.groupBy("status").count().show()
raw_df.filter(F.col("quantity") <= 0).show()
```

**Expected result:** Status counts: Delivered = 7, Cancelled = 1. The invalid-quantity query returns order 107.

# LAB 05 | Build Bronze, Silver and Gold Delta tables

**HANDS-ON WALKTHROUGH**

Estimated time: 45 minutes

**Objective:** Persist raw, clean and aggregated data as managed Delta tables.

01. In `01_load_bronze`, create the training schema under your **current catalog**. Use a different permitted schema if the CREATE SCHEMA action is denied.
02. Write the raw DataFrame to the Bronze table. This is an intentionally repeatable training write.
03. Create a second notebook named `02_transform_silver` and copy the Silver code.
04. Create a third notebook named `03_publish_gold` and copy the Gold code.
05. Execute all three notebooks in sequence, then verify the row counts.

## Bronze notebook - 01_load_bronze

**CODE - PYTHON**

```
spark.sql("CREATE SCHEMA IF NOT EXISTS dbx_food_lab")
(raw_df.write.format("delta").mode("overwrite")
      .saveAsTable("dbx_food_lab.orders_bronze"))
print("Bronze:", spark.table("dbx_food_lab.orders_bronze").count())
```

**NOTE** Keep the raw data-generation cell in 01_load_bronze. When scheduled as a job, this notebook must run both the data-generation and Bronze-write cells. Overwrite is for this repeatable training lab, not an automatically recommended production ingestion strategy.

## Silver notebook - 02_transform_silver

**CODE - PYTHON**

```
from pyspark.sql import functions as F
bronze = spark.table("dbx_food_lab.orders_bronze")
silver = (bronze.filter((F.col("status") == "Delivered") &
                        (F.col("quantity") > 0))
               .dropDuplicates(["order_id"])
               .withColumn("amount", F.round(
                   F.col("quantity") * F.col("unit_price"), 2)))
(silver.write.format("delta").mode("overwrite")
       .saveAsTable("dbx_food_lab.orders_silver"))
print("Silver:", silver.count())
```

## Gold notebook - 03_publish_gold

**CODE - PYTHON**

```
from pyspark.sql import functions as F
silver = spark.table("dbx_food_lab.orders_silver")
gold = (silver.groupBy("city")
        .agg(F.count("order_id").alias("delivered_orders"),
             F.round(F.sum("amount"), 2).alias("revenue")))
(gold.write.format("delta").mode("overwrite")
     .saveAsTable("dbx_food_lab.orders_gold_city"))
display(gold.orderBy("city"))
```

| Layer | Table | Expected count |
|---|---|---|
| Bronze | dbx_food_lab.orders_bronze | 8 |
| Silver | dbx_food_lab.orders_silver | 5 |
| Gold | dbx_food_lab.orders_gold_city | 3 cities |

## Validate rejected and duplicate records

**CODE - SQL**

```
%sql
SELECT order_id, status, quantity
FROM dbx_food_lab.orders_bronze
WHERE status <> 'Delivered' OR quantity <= 0;

SELECT order_id, COUNT(*) AS repeats
FROM dbx_food_lab.orders_bronze
GROUP BY order_id HAVING COUNT(*) > 1;
```

**Expected result:** The first query shows 103 (Cancelled) and 107 (negative quantity). The duplicate check shows order 106 twice. Three raw rows are excluded from the five-row Silver result.

**Production note:** This is an educational full-refresh pattern. Real pipelines typically define stable ingestion keys, incremental loads, quality rules, lineage and appropriate retention instead of overwriting every table on every run.

# LAB 06 | Query and visualize Gold data

**HANDS-ON WALKTHROUGH**

Estimated time: 30 minutes

**Objective:** Use SQL queries to produce a clean business answer and chart.

01. In a notebook create a SQL cell using `%sql`.
02. Run the city-revenue query below. Check the totals and ordering.
03. From the result grid choose a chart / visualization action if available; pick a bar chart with city on the x-axis and revenue on the y-axis.
04. Use the Catalog Explorer to inspect the Gold table columns and row data.
05. Capture the query result and optional chart as assessment evidence.

**CODE - SQL**

```
%sql
SELECT city, delivered_orders, revenue
FROM dbx_food_lab.orders_gold_city
ORDER BY revenue DESC;
```

| City | Delivered orders | Revenue (training units) |
|---|---|---|
| Chennai | 2 | 800.00 |
| Bengaluru | 1 | 500.00 |
| Coimbatore | 2 | 480.00 |

## Additional business query

**CODE - SQL**

```
%sql
SELECT SUM(delivered_orders) AS total_delivered_orders,
       ROUND(SUM(revenue), 2) AS total_revenue
FROM dbx_food_lab.orders_gold_city;
```

**Expected result: 5 delivered orders** and total revenue **1780.00**. This excludes cancelled, invalid and duplicate orders. Monetary values are synthetic units, not financial reporting.

# LAB 07 | Inspect Delta history and time travel

**HANDS-ON WALKTHROUGH**

Estimated time: 30 minutes

**Objective:** Use Delta table versions to compare before and after an update.

**01.** Open a new notebook called `04_delta_history` and choose Python.

**02.** Create or refresh the small demo table using the following first code block.

**03.** Record the current Delta version, then update status to Delivered.

**04.** Read the latest state and the snapshot from the saved version.

**05.** Open the table history output to identify the operations and timestamps.

**CODE - PYTHON**

```
table_name = "dbx_food_lab.order_audit_demo"
spark.sql(f"""CREATE OR REPLACE TABLE {table_name}
              USING DELTA AS
              SELECT 101 AS order_id, 'Placed' AS status""")
version_before = (spark.sql(f"DESCRIBE HISTORY {table_name}")
                  .selectExpr("max(version) AS v").first()["v"])
print("Version before update:", version_before)
```

**CODE - PYTHON**

```
spark.sql(f"""UPDATE {table_name}
              SET status = 'Delivered' WHERE order_id = 101""")
print("CURRENT STATE")
display(spark.table(table_name))
print("EARLIER STATE")
display(spark.sql(
    f"SELECT * FROM {table_name} VERSION AS OF {version_before}"))
display(spark.sql(f"DESCRIBE HISTORY {table_name}"))
```

**Expected result:** The current row shows **Delivered**; the earlier version shows **Placed**. The history includes at least the table write and update.

**NOTE** Time travel is limited by retained transaction logs and data files. It is not a replacement for backup. Do not run aggressive VACUUM retention settings during labs.

# LAB 08 | Explore Unity Catalog governance

**HANDS-ON WALKTHROUGH**

Estimated time: 30 minutes

**Objective:** Identify catalog/schema/table hierarchy and understand read permissions.

01. In the sidebar click **Catalog** to open Catalog Explorer.
02. Locate the active catalog from Lab 2, then expand `dbx_food_lab`.
03. Select `orders_gold_city` and inspect its schema, metadata and permissions view.
04. Run the introspection commands below in a SQL cell.
05. If you are an authorized catalog/table owner, review the example GRANT commands. Do not grant permissions to arbitrary users.

**CODE - SQL**

```
%sql
SELECT current_catalog() AS catalog_name;
SHOW SCHEMAS;
DESCRIBE TABLE dbx_food_lab.orders_gold_city;
SHOW GRANTS ON TABLE dbx_food_lab.orders_gold_city;
```

## Privileged example - use only when authorized

**CODE - SQL - TEMPLATE, NOT RUNNABLE AS-IS**

```
-- Replace <catalog_name> and <group_name> with real values.
-- Run separately only if you own / manage the objects.
GRANT USE CATALOG ON CATALOG <catalog_name> TO `<group_name>`;
GRANT USE SCHEMA ON SCHEMA <catalog_name>.dbx_food_lab
  TO `<group_name>`;
GRANT SELECT ON TABLE <catalog_name>.dbx_food_lab.orders_gold_city
  TO `<group_name>`;
```

| Level | Example | Role in access |
|---|---|---|
| Catalog | workspace-specific catalog | Parent namespace; USE CATALOG |
| Schema | dbx_food_lab | Logical grouping; USE SCHEMA |
| Table | orders_gold_city | Read table content; SELECT |

**Expected result:** You understand that **USE CATALOG + USE SCHEMA + SELECT** are typically all required for querying a UC table, unless an applicable broader privilege or ownership applies.

**NOTE** The example GRANT statements contain placeholders. Skip the write-permission exercise if you do not have a permitted group or admin rights; the inspection tasks alone complete this lab.

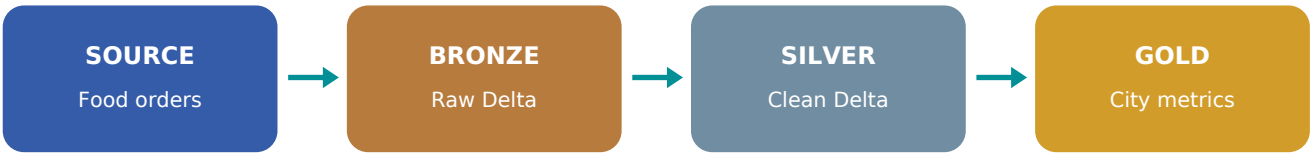
# LAB 09 | Create and run a three-task Lakeflow Job

**HANDS-ON WALKTHROUGH**

Estimated time: 40 minutes

**Objective:** Orchestrate notebooks with explicit dependencies and run them manually.

01. Confirm the following three notebooks run independently: `01_load_bronze`, `02_transform_silver`, and `03_publish_gold`.
02. Open **Jobs & Pipelines > Create > Job**. Name the job `Food_Orders_Medallion_Job`.
03. Add a **Notebook** task named `load_bronze`; choose Source = Workspace, then select notebook 01_load_bronze.
04. Add a second Notebook task `transform_silver`, selecting 02_transform_silver; set **Depends on = load_bronze**.
05. Add a third Notebook task `publish_gold`, selecting 03_publish_gold; set **Depends on = transform_silver**.
06. Choose serverless compute when offered; otherwise select an approved jobs compute resource. Save each task.
07. Click **Run now**, open the run details page, and inspect task status and outputs.
08. If you are conducting a shared lab, keep triggers manual until the workshop concludes.




Unity Catalog governs tables | Lakeflow Jobs orchestrates steps

**Expected result:** Three task tiles succeed in sequence. The Gold table contains 3 rows and the same revenue totals as Lab 6.

## All-purpose compute vs jobs compute

**All-purpose:** intended for interactive user sessions; usually stays active until terminated. **Jobs compute:** allocated for automated job tasks. **Serverless jobs:** Databricks handles compute provisioning. The task compute choices shown in your UI depend on workspace policies.

# LAB 10 | Monitor logs, force a lab error, and repair

**HANDS-ON WALKTHROUGH**

Estimated time: 40 minutes

**Objective:** Diagnose failed tasks without rerunning completed upstream tasks unnecessarily.

01. Open **Jobs & Pipelines**, select `Food_Orders_Medallion_Job`, open the **Runs** tab, then select the latest successful run.
02. Click each task. Observe elapsed time, task result, notebook output and any available diagnostic information.
03. In the `03_publish_gold` notebook, temporarily add the error line below as the **first Python cell**.
04. Trigger the job again with **Run now**. Observe that the Gold task fails (and Bronze/Silver should complete).
05. Remove the intentional error and save the notebook.
06. From the failed multi-task run, select **Repair run** and rerun the failed Gold task. If the repair action is not available, use a new job run instead.
07. Verify that the repaired run is successful and review the task output.

**CODE - PYTHON**

```
raise ValueError("Training-only failure: remove this line")
```

**Expected result:** You identify the failed notebook task, read the error, correct its code, and get a successful run. The Gold table is rebuilt with three rows.

**NOTE** Repair run is for jobs containing at least two tasks; a single-task job should be rerun. Repair is not available for every execution state or workspace feature set.

## Retry policy, not a fix for deterministic code bugs

In the task settings, look for an advanced **Retries** setting or serverless retry behavior. Retries can help temporary failures, but a hardcoded `ValueError` will keep failing until corrected. Configure retries sparingly, review job-level notifications, and document who owns failed runs.

# LAB 11 | Diagnose compute and Spark execution failures

**HANDS-ON WALKTHROUGH**

Estimated time: 30 minutes

**Objective:** Choose evidence before changing infrastructure settings.

| Observed symptom | First check | Likely next action |
|---|---|---|
| Notebook not running | Connected compute and permissions | Connect serverless or approved running compute. |
| Classic cluster stuck starting | Compute page > Event log | Read event reason: quota, permissions, network, or init script. |
| PySpark Python exception | Notebook cell error + stack trace | Fix code / types and rerun only required cells. |
| Job has FAILED task | Jobs > Runs > failed task | Read notebook output; inspect parameters and dependencies. |
| Permission denied | Catalog grants; active catalog | Request USE CATALOG, USE SCHEMA or table privilege. |
| Table not found | current_catalog(), SHOW SCHEMAS | Correct qualified table path and notebook order. |
| Slow Spark stage | Spark UI or query profile, if offered | Check skew, shuffle, input size, joins and partitions. |
| Free Edition quota exceeded | Workspace error / Free Edition limits | Resume only after quota reset or use an eligible environment. |

## Classic compute - where to find logs

01. Open **Compute**, select your cluster, and inspect the **Event log** for startup and termination events.
02. Review driver logs (**stdout**, **stderr**, Log4j) if the interface exposes them.
03. Open **Spark UI** and check stages and tasks when diagnosing execution performance.
04. Record the complete error message, timestamp, job/run ID and compute ID before changing configurations.

**Expected result:** You can distinguish code, access, input-data and infrastructure errors, and capture an error message with supporting job or compute context.

# LAB 12 | Capstone, checks and cleanup

**HANDS-ON WALKTHROUGH**

Estimated time: 30 minutes

**Objective:** Demonstrate a complete data workflow and collect evidence.

## Capstone exercise

01. Show your folder with 01_load_bronze, 02_transform_silver, 03_publish_gold, and 04_delta_history.
02. Show Bronze count = 8, Silver count = 5, Gold count = 3.
03. Show city revenues: Chennai = 800, Bengaluru = 500, Coimbatore = 480.
04. Show an earlier Delta snapshot for the audit demo and the DESCRIBE HISTORY output.
05. Show the job graph and a successful run. If allowed, show the intentionally failed run and successful repair.
06. Explain why governance and minimum necessary access should be configured at the catalog, schema and table levels.

**CODE - SQL**

```
%sql
SELECT 'Bronze' AS layer, COUNT(*) AS rows_count
FROM dbx_food_lab.orders_bronze
UNION ALL
SELECT 'Silver', COUNT(*) FROM dbx_food_lab.orders_silver
UNION ALL
SELECT 'Gold', COUNT(*) FROM dbx_food_lab.orders_gold_city;
```

## Submission checklist

| Evidence | Pass condition |
|---|---|
| Workspace and notebooks | Four named notebooks visible |
| Data quality | Duplicate, cancellation and negative quantity handled |
| Delta tables | Bronze 8 / Silver 5 / Gold 3 |
| Analytics | Five delivered orders; revenue 1780 |
| Time travel | Before update = Placed; now = Delivered |
| Job orchestration | 3 dependent tasks, final run successful |
| Operations | Know where to inspect job task errors and compute events |

# Trainer reference and further reading

## Suggested instructor-led timing

| Block | Time | Topics |
|---|---|---|
| Orientation | 45 min | Lab 1-2: workspace, notebooks, execution |
| Execution and data | 75 min | Lab 3-4: compute, Spark DataFrames, bad data |
| Delta / medallion | 100 min | Lab 5-7: Bronze, Silver, Gold, SQL, time travel |
| Governance and orchestration | 85 min | Lab 8-9: Unity Catalog and Jobs |
| Operations and capstone | 75 min | Lab 10-12: repair, logs, validation |

## Common teaching questions

**Q: Why Delta instead of raw CSV?** Delta supports managed table semantics, transactional changes and versioned history. CSV is a portable input file format, not by itself a transactional table.

**Q: Are all servers / cluster controls visible in Free Edition?** No. Free Edition uses managed serverless compute, with resource quotas and feature restrictions.

**Q: Why are Gold totals lower than Bronze implied sales?** Only valid, deduplicated delivered orders qualify as revenue in this simplified scenario.

**Q: Why do notebooks work individually but job tasks fail?** Jobs run in task order and may use different compute, identity, permissions or parameters. Each task must independently load its inputs.

**Q: Why does VERSION AS OF not use a fixed version number?** Prior runs and table replacement can alter the history. Capture a known version programmatically before an update.

## After the workshop

Stop unused classic compute, remove temporary schedules, and retain or delete lab tables according to your workspace policy. Do not drop shared schemas or delete data you do not own.

## Official documentation - checked November 2026

**Databricks Free Edition** - https://docs.databricks.com/aws/en/getting-started/free-edition

**Classic compute configuration** - https://docs.databricks.com/aws/en/compute/configure

**Create your first table** - https://docs.databricks.com/aws/en/getting-started/create-table

**Delta Lake tutorial** - https://docs.databricks.com/aws/en/delta/tutorial

**Work with table history** - https://docs.databricks.com/aws/en/tables/history

**Unity Catalog privileges** - https://docs.databricks.com/aws/en/data-governance/unity-catalog/access-control/privileges-reference

**Lakeflow Jobs quickstart** - https://docs.databricks.com/aws/en/jobs/jobs-quickstart

**Repair job failures** - https://docs.databricks.com/aws/en/jobs/repair-job-failures

**Compute event and driver logs** - https://docs.databricks.com/aws/en/compute/clusters-manage